

hyperparameters ex1:

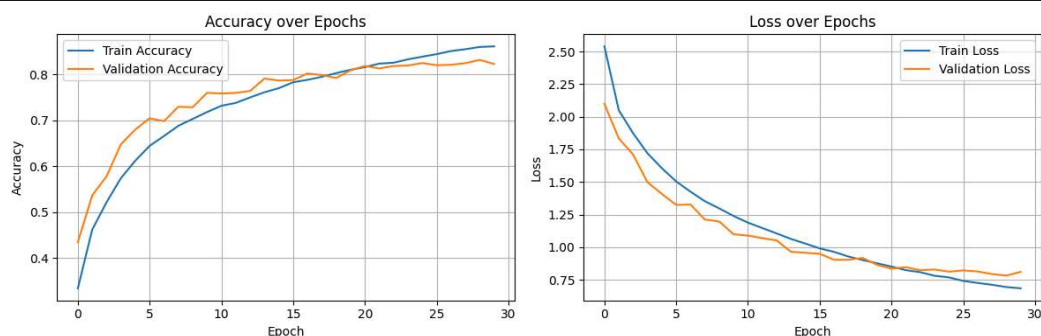
experiments:

1. SGD:

```
model.compile(optimizer='sgd', loss='categorical_crossentropy', metrics=['accuracy'])
```

Network: epochs = 30 batch_size = 32 optimizer= 'sgd'	<pre>layers.BatchNormalization(), layers.Conv2D(32, (3, 3), padding='same', activation='relu', kernel_regularizer=regularizers.l2(0.001)), layers.BatchNormalization(), layers.Conv2D(32, (3, 3), padding='same', activation='relu', kernel_regularizer=regularizers.l2(0.001)), layers.BatchNormalization(), layers.Dropout(0.2), layers.Conv2D(64, (3, 3), padding='same', activation='relu', kernel_regularizer=regularizers.l2(0.001)), layers.BatchNormalization(), layers.Conv2D(64, (3, 3), padding='same', activation='relu', kernel_regularizer=regularizers.l2(0.001)), layers.BatchNormalization(), layers.MaxPooling2D((2, 2)), layers.Dropout(0.3), layers.Conv2D(82, (3, 3), padding='same', activation='relu', kernel_regularizer=regularizers.l2(0.001)), layers.BatchNormalization(), layers.Conv2D(82, (3, 3), padding='same', activation='relu', kernel_regularizer=regularizers.l2(0.001)), layers.BatchNormalization(), layers.MaxPooling2D((2, 2)), layers.Dropout(0.4), layers.Flatten(), layers.Dense(95, activation='relu', kernel_regularizer=regularizers.l2(0.001)), layers.BatchNormalization(), layers.Dense(89, activation='relu', kernel_regularizer=regularizers.l2(0.001)), layers.BatchNormalization(), layers.Dropout(0.5), layers.Dense(num_classes, activation='softmax'),</pre>
Evaluation:	Test Loss: 0.828 Test Accuracy: 0.818 Validation Loss: 0.810 Validation Accuracy: 0.822 Training Loss: 0.456 Training Accuracy: 0.942

Visualization:



2. Momentum update:

```
model.compile(optimizer=SGD(momentum=0.95), loss='categorical_crossentropy', metrics=['accuracy'])
```

Network:
epochs = 30
batch_size = 128
optimizer=SGD(momentum=0.95)

```
layers.BatchNormalization(),  
layers.Conv2D(32, (3, 3), padding='same',  
activation='relu',kernel_regularizer=regularizers.l2(0.001)),  
layers.BatchNormalization(),  
layers.Conv2D(32, (3, 3), padding='same',  
activation='relu',kernel_regularizer=regularizers.l2(0.001)),  
layers.BatchNormalization(),  
layers.MaxPooling2D((2, 2)),  
layers.Dropout(0.15),
```

```
layers.Conv2D(64, (3, 3), padding='same',  
activation='relu',kernel_regularizer=regularizers.l2(0.001)),  
layers.BatchNormalization(),  
layers.Conv2D(64, (3, 3), padding='same',  
activation='relu',kernel_regularizer=regularizers.l2(0.001)),  
layers.BatchNormalization(),  
layers.MaxPooling2D((2, 2)),  
layers.Dropout(0.2),
```

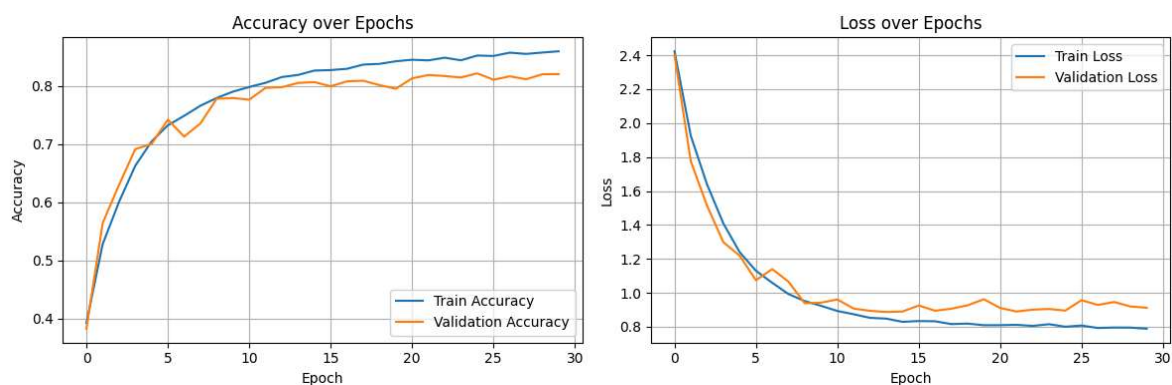
```
layers.Conv2D(82, (3, 3), padding='same',  
activation='relu',kernel_regularizer=regularizers.l2(0.001)),  
layers.BatchNormalization(),  
layers.Conv2D(82, (3, 3), padding='same',  
activation='relu',kernel_regularizer=regularizers.l2(0.001)),  
layers.BatchNormalization(),  
layers.MaxPooling2D((2, 2)),  
layers.Dropout(0.3),
```

```
layers.Flatten(),  
layers.Dense(95, activation='relu',kernel_regularizer=regularizers.l2(0.001)),  
layers.BatchNormalization(),  
layers.Dense(89, activation='relu',kernel_regularizer=regularizers.l2(0.001)),  
layers.BatchNormalization(),  
layers.Dropout(0.4),  
layers.Dense(num_classes, activation='softmax'),
```

Evaluation:

Test Loss: 0.909
Test Accuracy: 0.820
Validation Loss: 0.911
Validation Accuracy: 0.820
Training Loss: 0.647
Training Accuracy: 0.907

Visualization:

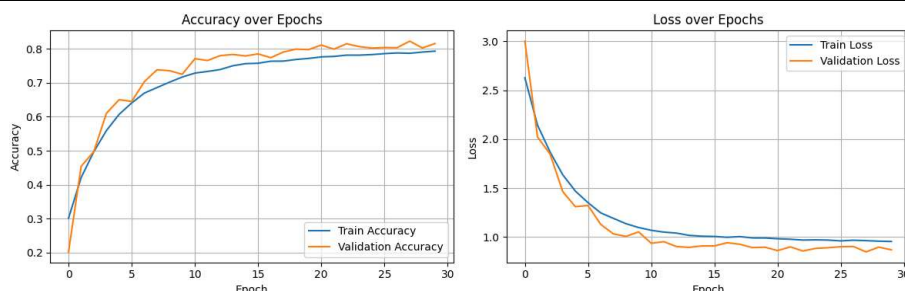


3. Nesterov Momentum update (nag):

```
model.compile(optimizer=keras.optimizers.SGD(momentum=0.9, nesterov=True),  
loss='categorical_crossentropy', metrics=['accuracy'])
```

Network: epochs = 30 batch_size = 128 optimizer=SGD(momentum=0.95, nesterov=True)	<pre>layers.BatchNormalization(), layers.Conv2D(32, (3, 3), padding='same', activation='relu',kernel_regularizer=regularizers.l2(0.001)), layers.BatchNormalization(), layers.Dropout(0.15), layers.Conv2D(32, (3, 3), padding='same', activation='relu',kernel_regularizer=regularizers.l2(0.001)), layers.BatchNormalization(), layers.MaxPooling2D((2, 2)), layers.Dropout(0.15), layers.Conv2D(68, (3, 3), padding='same', activation='relu',kernel_regularizer=regularizers.l2(0.001)), layers.BatchNormalization(), layers.Dropout(0.25), layers.Conv2D(68, (3, 3), padding='same', activation='relu',kernel_regularizer=regularizers.l2(0.001)), layers.BatchNormalization(), layers.MaxPooling2D((2, 2)), layers.Dropout(0.25), layers.Conv2D(82, (3, 3), padding='same', activation='relu',kernel_regularizer=regularizers.l2(0.001)), layers.BatchNormalization(), layers.Dropout(0.35), layers.Conv2D(82, (3, 3), padding='same', activation='relu',kernel_regularizer=regularizers.l2(0.001)), layers.BatchNormalization(), layers.MaxPooling2D((2, 2)), layers.Dropout(0.35), layers.Flatten(), layers.Dense(95, activation='relu',kernel_regularizer=regularizers.l2(0.001)), layers.BatchNormalization(), layers.Dropout(0.45), layers.Dense(89, activation='relu',kernel_regularizer=regularizers.l2(0.001)), layers.BatchNormalization(), layers.Dropout(0.45), layers.Dense(num_classes, activation='softmax'),</pre>
Evaluation:	Test Loss: 0.879 Test Accuracy: 0.817 Validation Loss: 0.868 Validation Accuracy: 0.816 Training Loss: 0.751 Training Accuracy: 0.858

Visualization:

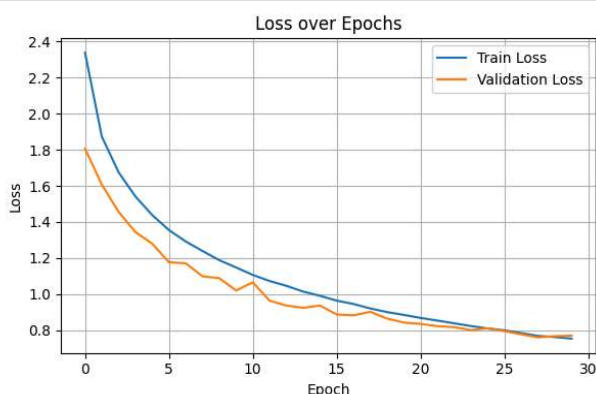
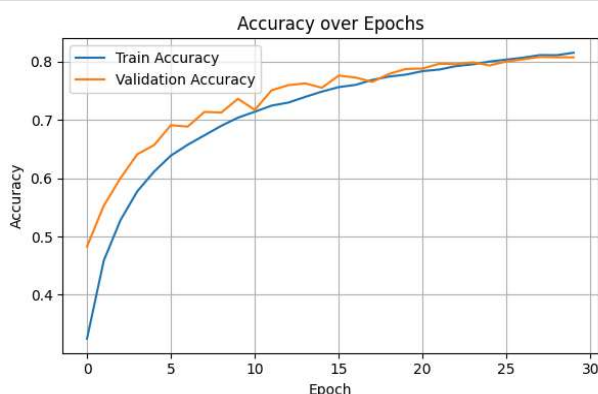


4. AdaGrad update:

```
model.compile(optimizer=Adagrad(learning_rate=0.1), loss='categorical_crossentropy', metrics=['accuracy'])
```

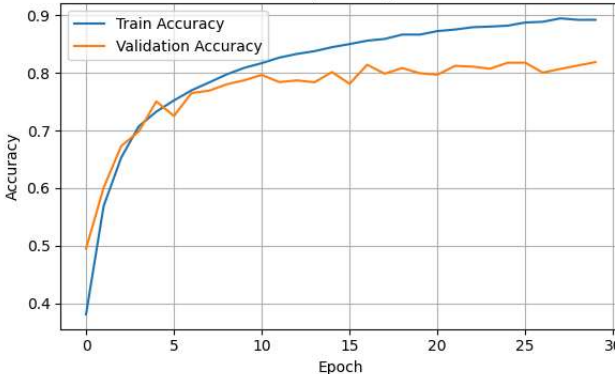
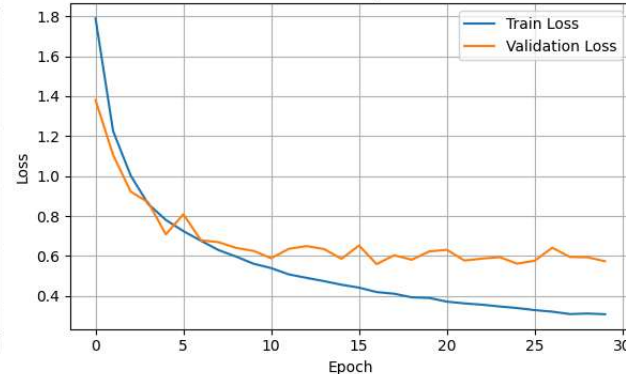
Network: epochs = 30 batch_size = 32 learning_rate=0.01	<pre> layers.BatchNormalization(), layers.Conv2D(32, (3, 3), activation='relu',kernel_regularizer=regularizers.l2(0.001)), layers.BatchNormalization(), layers.Conv2D(32, (3, 3), activation='relu', padding='same',kernel_regularizer=regularizers.l2(0.001)), layers.BatchNormalization(), layers.Conv2D(32, (3, 3), activation='relu', padding='same',kernel_regularizer=regularizers.l2(0.001)), layers.BatchNormalization(), layers.MaxPooling2D((2, 2)), layers.Dropout(0.2), layers.Conv2D(46, (3, 3), activation='relu',kernel_regularizer=regularizers.l2(0.001)), layers.BatchNormalization(), layers.Conv2D(46, (3, 3), activation='relu', padding='same',kernel_regularizer=regularizers.l2(0.001)), layers.BatchNormalization(), layers.Conv2D(46, (3, 3), activation='relu', padding='same',kernel_regularizer=regularizers.l2(0.001)), layers.BatchNormalization(), layers.Dropout(0.3), layers.Conv2D(68, (3, 3), activation='relu',kernel_regularizer=regularizers.l2(0.001)), layers.BatchNormalization(), layers.Conv2D(68, (3, 3), activation='relu', padding='same',kernel_regularizer=regularizers.l2(0.001)), layers.BatchNormalization(), layers.Conv2D(68, (3, 3), activation='relu', padding='same',kernel_regularizer=regularizers.l2(0.001)), layers.BatchNormalization(), layers.MaxPooling2D((2, 2)), layers.Dropout(0.4), layers.Flatten(), layers.Dense(89, activation='relu'), layers.BatchNormalization(), layers.Dense(95, activation='relu'), layers.BatchNormalization(), layers.Dropout(0.5), layers.Dense(10, activation='softmax') </pre>
Evaluation:	Test Loss: 0.799 Test Accuracy: 0.799 Validation Loss: 0.770 Validation Accuracy: 0.808 Training Loss: 0.533 Training Accuracy: 0.888

Visualization:



5. Adadelat update:

```
model.compile(optimizer=Adadelat(learning_rate=0.01), loss='categorical_crossentropy',
metrics=['accuracy'])
```

Network: epochs = 30 batch_size = 32 optimizer= Adadelat(learning_rate = 0.5)	<pre>layers.Conv2D(32, (3, 3), activation='relu'), layers.BatchNormalization(), layers.Dropout(0.1), layers.Conv2D(32, (3, 3), activation='relu', padding='same'), layers.BatchNormalization(), layers.Dropout(0.1), layers.MaxPooling2D((2, 2)), layers.Conv2D(68, (3, 3), activation='relu'), layers.BatchNormalization(), layers.Dropout(0.15), layers.Conv2D(68, (3, 3), activation='relu', padding='same'), layers.BatchNormalization(), layers.Dropout(0.15), layers.MaxPooling2D((2, 2)), layers.Conv2D(95, (3, 3), activation='relu'), layers.BatchNormalization(), layers.Dropout(0.2), layers.Conv2D(95, (3, 3), activation='relu', padding='same'), layers.BatchNormalization(), layers.Dropout(0.2), layers.MaxPooling2D((2, 2)), layers.Flatten(), layers.Dense(89, activation='relu'), layers.BatchNormalization(), layers.Dropout(0.4), layers.Dense(10, activation='softmax')</pre>																																																
Evaluation:	Test Loss: 0.599 Test Accuracy: 0.814 Validation Loss: 0.574 Validation Accuracy: 0.819 Training Loss: 0.176 Training Accuracy: 0.943																																																
Visualization:																																																	
Accuracy over Epochs <table><thead><tr><th>Epoch</th><th>Train Accuracy</th><th>Validation Accuracy</th></tr></thead><tbody><tr><td>0</td><td>0.38</td><td>0.50</td></tr><tr><td>5</td><td>0.75</td><td>0.75</td></tr><tr><td>10</td><td>0.82</td><td>0.80</td></tr><tr><td>15</td><td>0.85</td><td>0.80</td></tr><tr><td>20</td><td>0.88</td><td>0.82</td></tr><tr><td>25</td><td>0.90</td><td>0.82</td></tr><tr><td>30</td><td>0.94</td><td>0.82</td></tr></tbody></table> Loss over Epochs <table><thead><tr><th>Epoch</th><th>Train Loss</th><th>Validation Loss</th></tr></thead><tbody><tr><td>0</td><td>1.80</td><td>1.40</td></tr><tr><td>5</td><td>0.80</td><td>0.80</td></tr><tr><td>10</td><td>0.55</td><td>0.65</td></tr><tr><td>15</td><td>0.45</td><td>0.60</td></tr><tr><td>20</td><td>0.38</td><td>0.60</td></tr><tr><td>25</td><td>0.35</td><td>0.60</td></tr><tr><td>30</td><td>0.32</td><td>0.60</td></tr></tbody></table>		Epoch	Train Accuracy	Validation Accuracy	0	0.38	0.50	5	0.75	0.75	10	0.82	0.80	15	0.85	0.80	20	0.88	0.82	25	0.90	0.82	30	0.94	0.82	Epoch	Train Loss	Validation Loss	0	1.80	1.40	5	0.80	0.80	10	0.55	0.65	15	0.45	0.60	20	0.38	0.60	25	0.35	0.60	30	0.32	0.60
Epoch	Train Accuracy	Validation Accuracy																																															
0	0.38	0.50																																															
5	0.75	0.75																																															
10	0.82	0.80																																															
15	0.85	0.80																																															
20	0.88	0.82																																															
25	0.90	0.82																																															
30	0.94	0.82																																															
Epoch	Train Loss	Validation Loss																																															
0	1.80	1.40																																															
5	0.80	0.80																																															
10	0.55	0.65																																															
15	0.45	0.60																																															
20	0.38	0.60																																															
25	0.35	0.60																																															
30	0.32	0.60																																															

6. RMSProp update:

```
model.compile(optimizer='rmsprop',loss='categorical_crossentropy',metrics=['accuracy'])
```

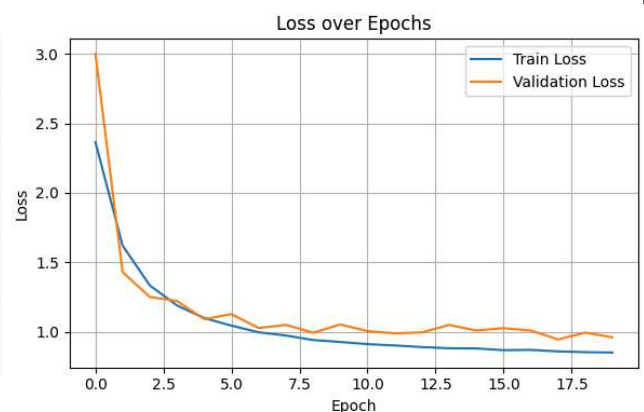
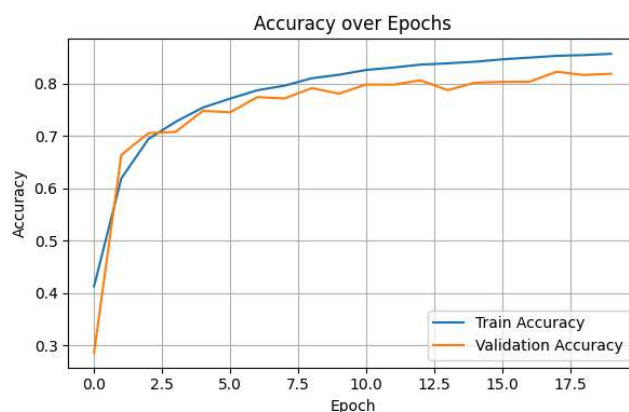
Network:
epochs = 20
batch_size = 128

```
layers.BatchNormalization(),  
layers.Conv2D(32, (3, 3), padding='same', activation='relu',  
kernel_regularizer=regularizers.l2(0.001)),  
layers.BatchNormalization(),  
layers.Conv2D(32, (3, 3), padding='same', activation='relu',  
kernel_regularizer=regularizers.l2(0.001)),  
layers.BatchNormalization(),  
layers.Dropout(0.2),  
  
layers.Conv2D(64, (3, 3), padding='same', activation='relu',  
kernel_regularizer=regularizers.l2(0.001)),  
layers.BatchNormalization(),  
layers.Conv2D(64, (3, 3), padding='same', activation='relu',  
kernel_regularizer=regularizers.l2(0.001)),  
layers.BatchNormalization(),  
layers.MaxPooling2D((2, 2)),  
layers.Dropout(0.3),  
  
layers.Conv2D(82, (3, 3), padding='same', activation='relu',  
kernel_regularizer=regularizers.l2(0.001)),  
layers.BatchNormalization(),  
layers.Conv2D(82, (3, 3), padding='same', activation='relu',  
kernel_regularizer=regularizers.l2(0.001)),  
layers.BatchNormalization(),  
layers.MaxPooling2D((2, 2)),  
layers.Dropout(0.4),  
  
layers.Flatten(),  
layers.Dense(95, activation='relu', kernel_regularizer=regularizers.l2(0.001)),  
layers.BatchNormalization(),  
layers.Dense(89, activation='relu', kernel_regularizer=regularizers.l2(0.001)),  
layers.BatchNormalization(),  
layers.Dropout(0.5),  
layers.Dense(num_classes, activation='softmax'),
```

Evaluation:

Test Loss: 0.989
Test Accuracy: 0.813
Validation Loss: 0.961
Validation Accuracy: 0.819
Training Loss: 0.698
Training Accuracy: 0.910

Visualization:



7. Adam update:

```
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

Network:
epochs = 15
batch_size = 128

```
layers.BatchNormalization(),  
layers.Conv2D(32, (3, 3), activation='relu', padding='same',  
kernel_regularizer=regularizers.l2(0.001)),  
layers.BatchNormalization(),  
layers.Conv2D(32, (3, 3), padding='same', activation='relu',  
kernel_regularizer=regularizers.l2(0.001)),  
layers.BatchNormalization(),  
layers.MaxPooling2D((2, 2)),  
layers.Dropout(0.2),  
  
layers.Conv2D(68, (3, 3), activation='relu', padding='same',  
kernel_regularizer=regularizers.l2(0.001)),  
layers.BatchNormalization(),  
layers.Conv2D(68, (3, 3), padding='same', activation='relu',  
kernel_regularizer=regularizers.l2(0.001)),  
layers.BatchNormalization(),  
layers.MaxPooling2D((2, 2)),  
layers.Dropout(0.3),  
  
layers.Conv2D(95, (3, 3), activation='relu',  
kernel_regularizer=regularizers.l2(0.001)),  
layers.BatchNormalization(),  
layers.Conv2D(95, (3, 3), padding='same', activation='relu',  
kernel_regularizer=regularizers.l2(0.001)),  
layers.BatchNormalization(),  
layers.Dropout(0.4),  
  
layers.Flatten(),  
layers.Dense(89, activation='relu', kernel_regularizer=regularizers.l2(0.001)),  
layers.BatchNormalization(),  
layers.Dense(82, activation='relu', kernel_regularizer=regularizers.l2(0.001)),  
layers.BatchNormalization(),  
layers.Dropout(0.5),  
layers.Dense(num_classes, activation='softmax'),
```

Evaluation:

Test Loss: 0.966
Test Accuracy: 0.802
Validation Loss: 0.963
Validation Accuracy: 0.802
Training Loss: 0.762
Training Accuracy: 0.865

Visualization:

